external configuration files inside the codebase instead. This makes it far easier to see what's happening. The following code defines the model class for a project. Don't worry about the details for now. Instead, think about how much information is being expressed in a few lines of code:

```
class Project < ApplicationRecord
  belongs_to :portfolio

  has_one    :project_manager
  has_many   :milestones
  has_many   :deliverables, through: milestones

  validates  :name, :description, presence: true
  validates  :non_disclosure_agreement, acceptance: true
  validates  :short_name, uniqueness: true
end
```

A major philosophical underpinning of Rails that keeps code short and readable is the DRY principle, which stands for Don't Repeat Yourself (see *The Pragmatic Programmer, 20th Anniversary Edition* [Hun19]). Every piece of knowledge in a system should be expressed in one place. Rails uses the power of Ruby to bring that to life. You'll find little duplication in a Rails application; you say what you need to say in one place—a place often suggested by the conventions of the MVC architecture—and then move on. For programmers used to other web frameworks, where a simple change to the database schema could involve a dozen or more code changes, this was a revelation—and it still is.

From that principle, Rails is founded on the Rails Doctrine,[1] which is a set of nine pillars that explain why Rails works the way it does and how you can be most successful in using it. Not every pillar is relevant when just starting out with Rails, but one pillar in particular is most important: convention over configuration.

Convention over configuration means that Rails has sensible defaults for just about every aspect of knitting together your application. Follow the conventions, and you can write a Rails application using less code than a typical JavaScript application uses in JSON configuration. If you need to override the conventions,